

Getting Started with Stata

Workshop 1 · 75 minutes · No statistics background assumed. No prior Stata assumed.

Setting up

1. **Get the files.** Go to tinyurl.com/ndpop-stata-1 → the green **Code** button → **Download ZIP**. Unzip it somewhere you can find again.
2. **Stata is already installed on the lab workstations.** Your own laptop works too, if Stata is already on it. Everything today works the same on Windows and Mac. If you need help installing it, ask me afterwards.
3. **Double-click `first_looks.dta`** inside the folder you just unzipped. **That is how you open Stata today.** Opening the data opens the program, with the data already loaded. Section 2.9 comes back to what that did for you. **Lab workstations are shared machines.** Before you leave, copy your work to a USB stick, your email, or a cloud drive. Block 4 is yours to take home, so this matters more than usual.

How this sheet works

Type the monospace lines exactly as printed.

The three blocks hand you less and less, on purpose. Block 2 is at the Command window, and every command you type is printed here. Block 3 moves into a do-file: `block3.do` has the code already written, and you run it a line at a time, working out what each line did. Block 4 is the same, as time permits.

Most of this sheet tells you what to notice. Three places ask you to stop and work something out first: the prediction at 2.5, the failure at 3.3, and the question that closes the session.

The General Workflow

COLLECT → INSPECT ↔ CLEAN → ANALYZE → REPORT

COLLECT	not today: the data was extracted for you
INSPECT	Block 2 in this room
CLEAN	Block 3 in this room
ANALYZE	Block 4 if time permits
REPORT	Block 4 if time permits

INSPECT and CLEAN are a loop, not two steps in a row. Block 3 is where you run that loop for real.

COLLECT is the one we skip today. The Google Trends export and the Census extract were prepared before you arrived.

ANALYZE is learned in courses and across a degree. You will only have brief exposure here.

Block 2 First look at Stata inspect

Work in the Command window (bottom center panel).

2.1 Open it

The data is already open: double-clicking it started Stata. Your first command:

```
browse
```

One row is one state. There are 50, from the 1980 US Census. One column is one thing measured about every state.

`browse` is read-only. Look, do not edit. Close the window to get back to the Command window.

Keep the row question in mind. It comes back in Block 4, and Workshop 2 is built on it.

2.2 The windows

Nothing to type. **Results** is what Stata said. **Command** is where you type. **History** is everything you have run; Block 2 ends by turning it into a file. **Variables** lists what is in the data, and double-clicking a name types it for you. **Properties** shows what Stata knows about the variable you have selected.

Use a drop-down menu with the **Results** window in view. Stata writes out the command it just ran. That is how you find a command name without searching for it.

2.3 Two small experiments

```
Browse
bro
```

Capitals matter. Stata is case-sensitive, and **Browse** is not a command.

Most commands have a legal short form: convenient to type, less convenient to read later.

Your first error. Errors are normal, but frustrating. Welcome to “debugging.” *Read them to fix them.*

2.4 Describing, and the syntax of a command

Run these one at a time and watch what changes.

```
describe
describe pop
describe pop-popurban
describe pop*
describe pop??p

codebook pop has_nd
inspect pop has_nd
summarize pop has_nd
```

Every Stata command follows syntax rules.

A varlist has shorthands. `pop-popurban` is a range through the dataset’s own order, not an alphabetical one. `pop*` is a wildcard. `pop??p` matches one character per `?`.

Three ways of looking at the same variables. `codebook` takes them one at a time, in detail. `inspect` shows where the values sit and whether any are missing. `summarize` gives the summary statistics and nothing else.

2.5 Listing rows, and if

```
list state has_nd
list state has_nd if has_nd == 1
```

Check: Indiana, and only Indiana. `==` asks a question; `=` assigns a value. They are not interchangeable.

Predict first. Write down what you think each of the next two returns. Then run them.

```
list state has_nd if has_nd != 0
list state has_nd if has_nd > 0
```

Were you right? If not, why not?

Stata treats missing as larger than any number. That is arbitrary, and it is worth knowing.

```
help list
```

2.6 Counting

```
tabulate border
tabulate state border
tabulate border coastal
tabulate border coastal, row column

count if border == 1 & coastal == 1
list state if border == 1 & coastal == 1
```

Check: 35 and 15 on the first one. The second fills the screen with 50 rows, which is the lesson about when a cross-tabulation stops being useful.

Check: 7 states: Alaska, California, Maine, New Hampshire, New York, Texas, Washington.

Everything after the comma is an option.

2.8 Your first do-file

This is the point of the block, and it is done by clicking.

1. **History** window → the circular warning icon, which hides the commands that errored → **Copy All**
2. `doedit` opens an empty do-file editor
3. Paste
4. Save it as `first_do.do`
5. `do first_do.do`

That turns hours of interactive typing into something you can run again in seconds.

If someone handed you that file tomorrow, they could not follow it. It is a wall of commands with no explanation, including your typos and dead ends. What is missing is comments. That is what `block3.do` opens with.

2.9 Where has Stata been working all this time?

```
cls
pwd
```

Read what it says. You did not tell Stata where to work, but it has an answer: double-clicking a file chose the folder for you. Watch what happens when nothing chooses. **Close Stata completely** and start it again from the **Windows Start menu**, or **Spotlight** on a Mac. Do not double-click anything.

```
pwd
```

Check: somewhere else entirely: whatever folder Stata starts in. Your data is not there, and nothing you type will find it until you say where to look.

Fix: Open your unzipped folder in File Explorer or Finder, click the address bar, and copy the path. Type `cd "`, paste the path, and close the quote. It will look like `cd "C:/Users/nd_netid/Downloads/stata-onboarding"`. This is the one line on this sheet you do not type exactly as printed: the path is yours. Then check it:

```
pwd
```

Check: `pwd` now prints your workshop folder. **Use forward slashes.** They work on Windows and Mac both, and Windows hands you backslashes when you paste, so change them.

Block 3 Your first Stata project clean

Open `block3.do`. The commands are already written. Run them **a line at a time**: highlight the line and press **Ctrl-D** (Windows) or **Cmd-Shift-D** (Mac). Work out what each one did before you move on. This sheet numbers the steps and says what to look for.

3.1 Where Stata is working

1. Run `pwd`. If it is not your workshop folder, `cd` to it as you did in 2.9. Everything after that is a bare filename.

3.2 What a do-file starts with

2. Run the lines under the `use`. They are Stata's three ways of writing a comment, and they demonstrate themselves. `*` comments a whole line. `//` can follow a command on the same line. `/* */` spans several. Watch which variables `desc state` `// pop` describes.

Stata ignores comments; people read them. Put your name and today's date at the top of `block3.do`.

3.3 Does the data add up?

There are four age variables: `pop1t5`, `pop5_17`, `pop18p`, `pop65p`. Add them up and check the sum against `pop`.

3. Run the `generate` and `summarize` lines.

Check: the means differ: 4,518,149 against 5,027,652.

Your eyes can catch that here. With a bigger dataset it is harder.

4. Run the `assert`:

```
assert pop == check_pop
```

This fails on purpose, and it stops the do-file. That halt is the point of `assert`. A summary table is easy to skim past. A file that refuses to continue is not.

Why did it fail? Look at what each of the four age variables covers. Work it out before you run the next line.

5. Then continue. The `replace` redefines `check_pop` using only the groups that do not overlap, and the `assert` runs again.

Check: silence. `assert` prints nothing at all when it passes.

6. Run the rest of the section: the same check from the other side, `pop18p` minus `pop65p`. **That name is wrong.** The group runs 18 to 64, not 18 to 65, so the next line `renames` it. Then it builds `check2_pop`, asserts it, and drops both check variables.

3.7 Labels

Three kinds, broadest to narrowest: the dataset, a variable, and the values inside a variable.

7. Run the `label data` and `label variable` lines. They give the dataset a title and `coastal` a readable name.

8. Value labels work differently: you define a named set once, then apply it to as many variables as you like. Run the

tabulate, then the label define and label values, then the same tabulate again.

Check: same numbers, readable table.

The labels changed; the data did not. Labels sit on top of the values. The numbers underneath have not moved.

3.8 Groups

9. Run the first four lines and look at what they made. `_n` is *this row's number* and `_N` is *the total row count*, repeated identically on every row. Then they are dropped again.

10. Run the `bysort region:` lines. `bysort region:` in front of a command makes Stata run it once per region, which changes what `_n` and `_N` refer to. The last line lists one row per region instead of all 50.

Check: four rows: 9, 12, 16 and 13 states. The totals print in scientific notation, because they are raw counts of people. `format region_pop %15.0fc` makes them readable.

Inside `bysort region:`, `_N` counts the rows in *that region*, not in the dataset. `if nval == 1` keeps the first row of each region, so four rows come back instead of fifty. Workshop 2's `collapse` is built on this idea.

3.9 Log files

The do-file is what you asked. The log is what Stata answered.

11. Run the log section. It opens `test_log.txt`, prints two things to it, and closes it again.

Check: `test_log.txt` appears in your folder and opens as readable text.

Three small protections are doing real work in those two lines. `capture` stops a failed `log close` from halting the file, so you can run it twice. `, text` keeps the log readable; without it you get a screenful of `{smc1}` markup. `, replace` lets the second run overwrite the first.

12. Run the `save`. Your cleaned data becomes `first_looks_clean.dta`.

Block 4 First output from raw input yours to take home

We work through Block 4 as time permits. It closes the session: one figure on the screen, made by `block4.do`. The file is written to be followed on your own, from cold. Nothing in it depends on anything done during the session. It takes a raw file: a Google Trends export of search interest for Notre Dame, MIT and Clemson, 272 monthly rows from 2004 to 2026. It carries that to a figure you could put in a paper. Seven steps:

1. **Import** the CSV and inspect what arrived. *272 observations, 4 variables.*
2. **Fix the dates.** They arrive as text and are useless until converted. *It should read 01jan2004, not 16071.*
3. **Rename and label.** One variable imports as `massachusettsinstituteoftechnolo`, truncated at 32 characters.
4. **Declare it a time series** with `tsset`, or the line graphs will not draw. *2004m1 to 2026m8, delta: 1 month.*
5. **Analyze** with `correlate`, then `regress`.
6. **Draw it** with `scatter`, `twoway` and `tsline`, working up from the plain version to the layered one.
7. **Export** a figure. *A .png appears in your folder.*

In Block 2, one row was one state. What is one row here?

Block 5 Where to go next

Three things to look up: macros (`help macro`) · loops (`help foreach`) · user-written commands (`ssc install`)

Getting unstuck, in order: `help commandname` → search keyword → the internet, pasting the exact error text with the `r(###)` included → a person.

People who will help you:

- **ND Pop Stata Helpdesk.** Drop in, no appointment. **Mon–Thu 5–7pm, 4020D Jenkins Nanovic Halls.** Bring your laptop. populationanalytics.nd.edu/resources/stata-support-services
- **Navari Family Center for Digital Scholarship.** Hesburgh Library, 2nd floor NE corner. Consultations on data analysis, visualisation, GIS and text mining. cds@nd.edu

Online: the `help` command, which is the full manuals, already installed · stata.com/support · blog.stata.com · stats.oarc.ucla.edu/stata · stata.com/flyers/stata-cheat-sheets, free PDFs including one for syntax.

AI chatbots are useful for finding a command name or decoding an error. They also invent options that do not exist, fluently. **Do not run a Stata command from a chatbot until you understand it independently.**

A helpful (advanced) text: Cameron & Trivedi, *Microeconometrics Using Stata*, 2nd ed. (Stata Press, 2022).